# Design and simulation of a Multi-Algorithm

# Hash Engine

SHA-256, SHA-512 and SHA3-256 compression cores written in Verilog and verified by simulation

November 2026

## Document summary:

This project implements a hardware hashing engine featuring a top-level hash_engine module (handling controller logic, padding, and digest formatting) alongside three compression cores: sha256_core (u_s256), sha512_core (u_s512), and sha3_256_core (u_kec). Built to comply with NIST FIPS 180-4 (SHA-2) and FIPS 202 (SHA-3) standards across its source files (hash_engine.v, sha256_core.v, sha512_core.v, sha3_256_core.v, and tb_hash_engine.v available on https://github.com/djamal3010M/Crypto_Core/tree/main/RTL/HASH), the design is verified using a self-checking testbench (tb_hash_engine) that processes the message "algeria" and generates VCD waveform outputs viewable in GTKWave.

# Contents

1 Abstract
2 Introduction and Scope
3 Algorithm Background
4 Mathematical Foundation
5 System Architecture of hash_engine
6 Compression Core Designs
7 Verification and Simulation Results
8 Conclusion

# 1. Abstract

This report documents a complete hardware hashing subsystem written in Verilog. The subsystem, **hash_engine**, offers one streaming interface for three algorithms: SHA-256, SHA-512 and SHA3-256. Behind that interface sit three iterative compression cores that each execute one round per clock cycle: **sha256_core** (64 rounds), **sha512_core** (80 rounds) and **sha3_256_core** (24 Keccak rounds). The controller buffers incoming bytes, applies the correct padding and length encoding for each algorithm, starts the selected core once per block, chains multi-block messages, and publishes the digest.

the testbench and a formal mathematical specification, which forms the technical foundation. It describes the architecture of every module, interprets three GTKWave captures. The testbench hashes the seven-byte message "algeria" with all three algorithms and compares each result with an expected value. Those expected values were independently computed with Python's `hashlib` and agree exactly with the digests seen in the waveforms.

# 2. Introduction and Scope

## 2.1 Background and motivation

Cryptographic hash functions are essential for smart card systems, supporting digital signatures, secure authentication, key derivation, and transaction integrity checking. While software implementations offer flexibility, they can be slow and resource-intensive on embedded smart card microcontrollers, particularly for algorithms like SHA-512, whose 64-bit words are not suitable to handle efficiently on constrained processor architectures. Integrating a dedicated hardware engine offloads these computations, ensuring power-efficient execution and predictable latency. The hardware design tailored for this environment favors minimal silicon area and clarity over peak speed, utilizing an architecture where each core reuses a single round datapath for every round of a block to minimize the physical footprint on the chip.

## 2.2 Scope and objectives

The three compression cores process only complete, pre-padded blocks. Message buffering, padding, length encoding and block sequencing are the job of the top-level hash_engine. The objectives of this report are to:

- Describe the interface and internal organisation of the engine and of each core.
- Map the RTL to the algorithm specifications: message schedule, round function, feed-forward addition, sponge absorption, and padding.
- Verify the design by relating the testbench results and waveforms to the RTL cycle by cycle, and by cross-checking digests against a reference implementation.

# 3. Algorithm Background

SHA-256 and SHA-512 belong to the SHA-2 family (FIPS 180-4). They use a Merkle-Damgard construction: the padded message is split into fixed-size blocks, and a compression function updates a chaining value block by block. For each block the cores expand 16 input words into a longer message schedule, run a fixed number of rounds over eight working variables (a to h), and add the result to the previous chaining value modulo $2^w$, where w is the word size. SHA3-256 (FIPS 202) is built differently. It keeps a 1600-bit Keccak state split into a rate and a capacity. Each rate-sized block is XORed into the state, and the 24-round Keccak-f[1600] permutation is applied. The digest is read directly from the start of the state.

| Parameter | SHA-256 | SHA-512 | SHA3-256 |
|---|---|---|---|
| Standard | FIPS 180-4 | FIPS 180-4 | FIPS 202 |
| Construction | Merkle-Damgard | Merkle-Damgard | Sponge (Keccak) |
| Word / lane size | 32 bits | 64 bits | 64-bit lanes, 25 of them |
| Block / rate | 512 bits (64 B) | 1024 bits (128 B) | 1088 bits (136 B) |
| Digest size | 256 bits | 512 bits | 256 bits |
| Rounds per block | 64 | 80 | 24 |
| Schedule / state | 64 words, from 16 | 80 words, from 16 | 1600-bit state |
| Padding marker | 0x80 | 0x80 | 0x06 and 0x80 |
| Length field | 64 bits | 128 bits | none |
| Initial value | 6a09e667... | 6a09e667f3bcc908... | all-zero state |

*Table 1. Parameters of the three implemented algorithms*

The two SHA-2 cores are structurally almost identical: the same sliding-window schedule, the same round structure, and the same three-phase control. They differ in word width, round count, rotation constants and constant table. The SHA3 core has a different shape. It has no message schedule, no feed-forward addition and no separate finalisation cycle, because the digest is simply a view of the permuted state.

# 4. Mathematical Foundation

## 4.1 Purpose and conventions

This section gives the formal mathematical model of the three cores.

| Symbol | Meaning |
|---|---|
| $\wedge$ | bitwise AND |
| $\vee$ | Bitwise OR |
| $\neg$ | bitwise NOT |
| $\oplus$ | Bitwise XOR |
| >> | Shift right |
| << | Shift left |
| $ROTR^{n}, ROTL^{n}$ | Rotation right or left by n bits. |
| $SHR^{n}$ | Logical shift right by n bits. |
| \|\| | Concatenation of bit strings, most-significant part first. |
| $M^{(t)}$ | The t-th w-bit word of the current block; word 0 is the most-significant word of the block port. |
| $V_t$ | Working state ($a_t$, $b_t$, $c_t$, $d_t$, $e_t$, $f_t$, $g_t$, $h_t$) after t rounds. |
| $H^{(i)}$ | Chaining value after block i. $H^{(0)}$ is the standard initial value. |
| S, A[x,y] | Keccak state of 1600 bits, viewed as 25 lanes of 64 bits with lane index x + 5y. |

*Table 2. Notation used in the mathematical model*

## 4.2 SHA-256 compression model

All arithmetic is in the ring of 32-bit integers modulo $2^{32}$. The bitwise functions are the four rotation-based functions of FIPS 180-4 together with Ch(Choice) and Maj(Majority):

$$\mathrm{ROTR}^n(x) = (x \gg n) \vee (x \ll (32-n)) \tag{1}$$

$$\sigma_0(x) = \mathrm{ROTR}^7(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{SHR}^3(x) \tag{2}$$

$$\sigma_1(x) = \mathrm{ROTR}^{17}(x) \oplus \mathrm{ROTR}^{19}(x) \oplus \mathrm{SHR}^{10}(x) \tag{3}$$

$$\Sigma_0(x) = \mathrm{ROTR}^2(x) \oplus \mathrm{ROTR}^{13}(x) \oplus \mathrm{ROTR}^{22}(x) \tag{4}$$

$$\Sigma_1(x) = \mathrm{ROTR}^6(x) \oplus \mathrm{ROTR}^{11}(x) \oplus \mathrm{ROTR}^{25}(x) \tag{5}$$

$$\mathrm{Ch}(x,y,z) = (x \wedge y) \oplus (\neg x \wedge z) \tag{6}$$

$$\mathrm{Maj}(x,y,z) = (x \wedge y) \oplus (x \wedge z) \oplus (y \wedge z) \tag{7}$$

**Message schedule.** A padded 512-bit block is M = $M^{(0)}$ || $M^{(1)}$ ||... || $M^{(15)}$ with 32-bit words. The schedule word for round t is the input word for the first 16 rounds and a recurrence afterwards. In the RTL the recurrence reads the fixed window taps W[14], W[9], W[1] and W[0], which hold W(t-2), W(t-7), W(t-15) and W(t-16).

$$W_t = M^{(t)}, \qquad 0 \le t \le 15 \tag{8}$$

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16} \pmod{2^{32}}, \quad 16 \le t \le 63 \tag{9}$$

**Round function.** The initial state is $V_0 = H^{(i-1)}$, where $H^{(0)}$ is the constant H256 when `cont = 0` and the previous digest when `cont = 1`. For t = 0 to 63:

$$T_1^{(t)} = h_t + \Sigma_1(e_t) + \mathrm{Ch}(e_t, f_t, g_t) + K_t + W_t \pmod{2^{32}} \tag{10}$$

$$T_2^{(t)} = \Sigma_0(a_t) + \mathrm{Maj}(a_t, b_t, c_t) \pmod{2^{32}} \tag{11}$$

$$a_{t+1} = T_1^{(t)} + T_2^{(t)}, \quad e_{t+1} = d_t + T_1^{(t)} \pmod{2^{32}} \tag{12}$$

$$b_{t+1} = a_t, \quad c_{t+1} = b_t, \quad d_{t+1} = c_t \tag{13}$$

$$f_{t+1} = e_t, \quad g_{t+1} = f_t, \quad h_{t+1} = g_t \tag{14}$$

**Finalisation.** After 64 rounds the new chaining value is the word-wise sum of the initial chaining value and the final state $V_{64}$ = ($a_{64}$, ..., $h_{64}$):

$$H_j^{(i)} = H_j^{(i-1)} + V_{64,j} \pmod{2^{32}}, \quad j = 0, \ldots, 7 \tag{15}$$

## 4.3 SHA-512 compression model

The SHA-512 core uses the same structure over the ring of 64-bit integers modulo $2^{64}$, with 1024-bit blocks of sixteen 64-bit words and 80 rounds. Only the rotation amounts, the round constants $K_t$ (80 of them) and the modulus change:

$$\mathrm{ROTR}^n(x) = (x \gg n) \vee (x \ll (64-n)) \tag{16}$$

$$\sigma_0(x) = \mathrm{ROTR}^1(x) \oplus \mathrm{ROTR}^8(x) \oplus \mathrm{SHR}^7(x) \tag{17}$$

$$\sigma_1(x) = \mathrm{ROTR}^{19}(x) \oplus \mathrm{ROTR}^{61}(x) \oplus \mathrm{SHR}^6(x) \tag{18}$$

$$\Sigma_0(x) = \mathrm{ROTR}^{28}(x) \oplus \mathrm{ROTR}^{34}(x) \oplus \mathrm{ROTR}^{39}(x) \tag{19}$$

$$\Sigma_1(x) = \mathrm{ROTR}^{14}(x) \oplus \mathrm{ROTR}^{18}(x) \oplus \mathrm{ROTR}^{41}(x) \tag{20}$$

$$W_t = \sigma_1(W_{t-2}) + W_{t-7} + \sigma_0(W_{t-15}) + W_{t-16} \pmod{2^{64}}, \quad 16 \le t \le 79 \tag{21}$$

Ch, Maj and the round equations for $T_1$, $T_2$, a to h are identical in form to the SHA-256 equations, evaluated modulo $2^{64}$ for t = 0 to 79. The finalisation uses the state after 80 rounds:

$$H_j^{(i)} = H_j^{(i-1)} + V_{80,j} \pmod{2^{64}}, \quad j = 0, \ldots, 7 \tag{22}$$

## 4.4 Keccak-f[1600] sponge model

The state S has 1600 bits and is viewed as a 5 x 5 array of 64-bit lanes A[x,y] with x, y in {0, ..., 4}. Lane A[x,y] occupies S[64(x+5y) +: 64], which is the array index x + 5y used in the RTL. Byte i of the state is $S_{byte,i}$ = S[8i +: 8].

**Absorption.** For SHA3-256 the rate is 136 bytes. The state is first set according to **init** and then the block is XORed into the first 136 bytes. The input block is big-endian: byte i of the block is block[1151-8i -: 8]. The 64 capacity bytes are not touched. The core port is 1152 bits wide; hash_engine drives its low 64 bits with zeros and sets bs = 136.

$$S^{(0)} = 0^{1600} \quad \text{if init} = 1$$
$$S^{(0)} = S_{\text{prev}} \quad \text{if init} = 0 \tag{23}$$

$$S_{\text{byte},i} \leftarrow S^{(0)}_{\text{byte},i} \oplus M_{\text{byte},i}, \quad 0 \le i < 136 \tag{24}$$

**Round permutation.** Each of the 24 rounds (r = 0, ..., 23) applies five steps in sequence. Theta computes column parities and diffuses them:

$$C[x] = \bigoplus_{y=0}^{4} A[x,y] \tag{25}$$

$$D[x] = C[(x-1) \bmod 5] \oplus \mathrm{ROTL}^1(C[(x+1) \bmod 5]) \tag{26}$$

$$A[x,y] \leftarrow A[x,y] \oplus D[x] \tag{27}$$

*Rho* and *pi* rotate each lane by a fixed offset r[x,y] and move it to a new position. In the RTL these two steps are one table of 24 constant bit-slice assignments, so they cost no logic.

$$B[y, (2x+3y) \bmod 5] = \mathrm{ROTL}^{r[x,y]}(A[x,y]) \tag{28}$$

| r[x,y] | x = 0 | x = 1 | x = 2 | x = 3 | x = 4 |
|---|---|---|---|---|---|
| **y = 0** | 0 | 1 | 62 | 28 | 27 |
| **y = 1** | 36 | 44 | 6 | 55 | 20 |
| **y = 2** | 3 | 10 | 43 | 25 | 39 |
| **y = 3** | 41 | 45 | 15 | 21 | 8 |
| **y = 4** | 18 | 2 | 61 | 56 | 14 |

*Table 3. Rho rotation offsets (left rotations) implemented by the rho/pi assignments*

*Chi* is the only non-linear step. It combines each lane with its two neighbours along a row, and *Iota* breaks the symmetry between rounds by XORing the round constant RC(r), the 24-entry function `rc()` of the RTL, into lane A[0,0]:

$$A[x,y] = B[x,y] \oplus (\neg B[(x+1) \bmod 5, y] \wedge B[(x+2) \bmod 5, y]) \tag{29}$$

$$A[0,0] \leftarrow A[0,0] \oplus \mathrm{RC}(r) \tag{30}$$

$$S^{(r+1)} = \iota_r(\chi(\rho\pi(\theta(S^{(r)})))), \quad r = 0, \ldots, 23 \tag{31}$$

**Digest extraction.** After 24 rounds the digest is the first 32 bytes of the state $S^{(24)}$. in the RTL the bytes are read from the little-endian state and presented most-significant byte first, so the digest is defined bytewise:

$$\mathrm{Digest}[255 - 8i -: 8] = S^{(24)}_{\mathrm{byte},\,i}, \quad i = 0, \ldots, 31 \tag{32}$$

### 4.5 Padding and block count

The cores process complete blocks only, so the message length decides how many blocks the controller must form. Let L be the message length in bits. For SHA-2 the padded message is the message, a single 1 bit, k zero bits, and the length as a fixed-width integer, such that the total is a multiple of the block size; the subscripts 256 and 512 below name the two SHA-2 functions, and N is the number of blocks. For SHA3-256 the bytes 0x06 and 0x80 are placed after the message within the 1088-bit rate; when only one byte remains they merge into 0x86.

$$k_{256} = (447 - L) \bmod 512, \qquad N_{256} = \left\lceil \frac{L+65}{512} \right\rceil \tag{33}$$

$$k_{512} = (895 - L) \bmod 1024, \qquad N_{512} = \left\lceil \frac{L+129}{1024} \right\rceil \tag{34}$$

$$N_{\mathrm{SHA3}} = \left\lfloor \frac{L}{1088} \right\rfloor + 1 \tag{35}$$

## 5. System Architecture of hash_engine

### 5.1 Overview

hash_engine is the top-level controller. It presents one streaming interface for all three algorithms. The user selects an algorithm with an init command, then submits the message in chunks of up to 64 bytes, flagging the last chunk. The engine appends each chunk to an internal buffer, forms complete blocks, starts the matching core, and handles the final padding. Figure 1 shows the main blocks. All three cores are instantiated permanently and share the clock and reset; only the selected core ever receives a start pulse.

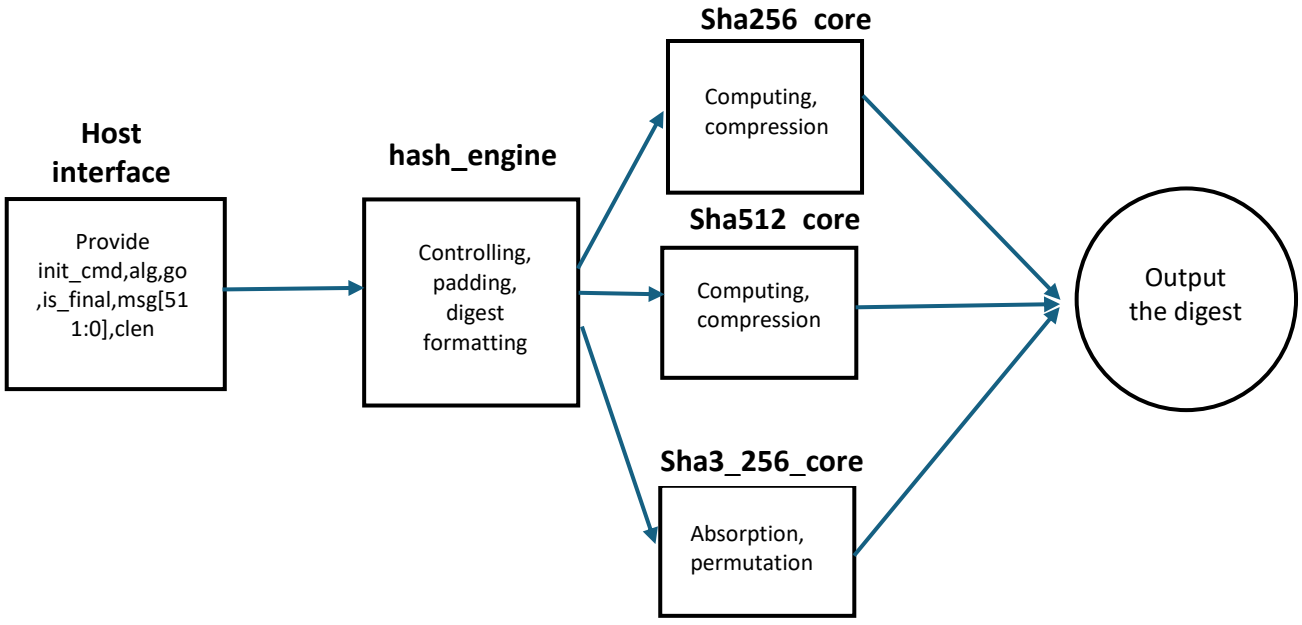



*Figure 1. Block diagram of hash_engine and its three compression cores*

## 5.2 Interface

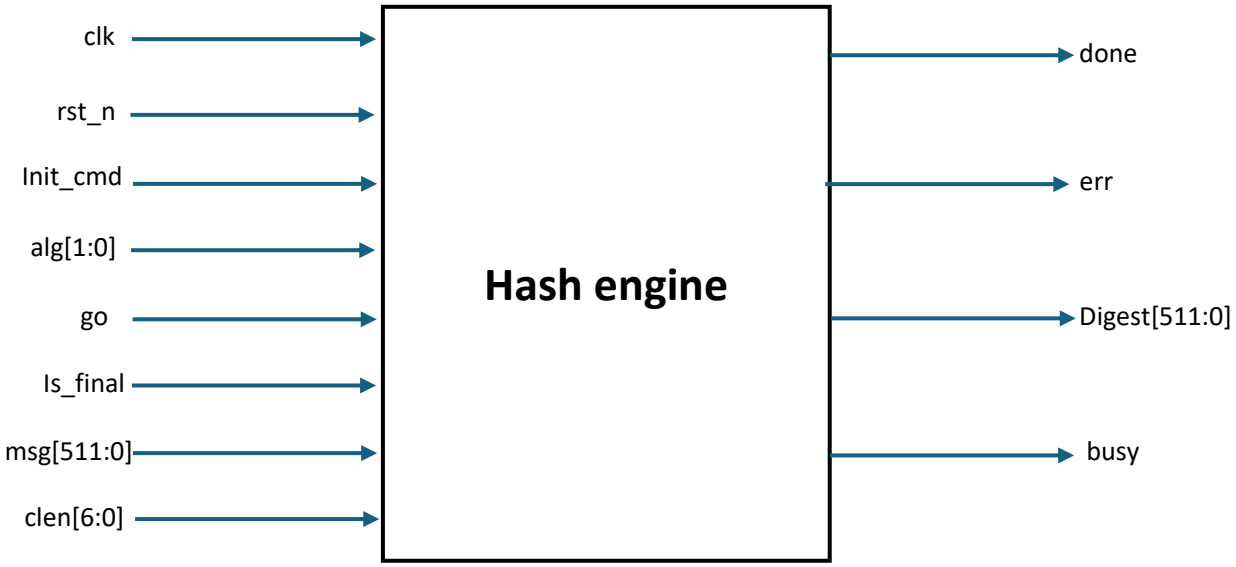



*Figure 2 Top-level I/O Block Diagram of the Hash engine*

| Port | Dir. | Width | Function |
|---|---|---|---|
| `clk` | in | 1 | Rising-edge clock for the controller and all three cores. |
| `rst_n` | in | 1 | Active-low asynchronous reset. |
| `init_cmd` | in | 1 | Starts a fresh session, clears buffered data and selects `alg`. Accepted only in S_IDLE. |
| `alg` | in | 2 | Algorithm code: 0 invalid, 1 SHA-256, 2 SHA-512, 3 SHA3-256. Sampled with init_cmd. |
| `go` | in | 1 | Submits one input chunk. Sampled in S_IDLE. |
| `is_final` | in | 1 | Marks the submitted chunk as the last chunk of the message. |
| `msg` | in | 512 | Up to 64 input bytes. |
| `clen` | in | 7 | Chunk length in bytes (0 to 64). |
| `done` | out | 1 | One-cycle pulse when a command is finished. After the final chunk it means the digest is ready. |
| `err` | out | 1 | Set for an invalid algorithm code, a chunk without a session, or clen above 64. |
| `digest` | out | 512 | Left-aligned digest. 256-bit digests occupy [511:256] and the low half is zero. |
| `busy` | out | 1 | High whenever the FSM is not in S_IDLE. |

*Table 4. Port list of hash_engine*

## 5.3 Chunk buffering

Incoming bytes are collected in a 1600-bit buffer `buf_r` whose valid bytes are always kept at the most-significant end. The size is chosen as the largest possible occupancy: one SHA3 rate block (136 bytes) that is not yet full plus one 64-byte chunk is at most 200 bytes, which is exactly 1600 bits. A byte mask removes any bits of `msg` beyond `clen`, and the chunk is shifted right by `fill` bytes and ORed into the buffer. The register `fill` counts valid bytes, and `total` accumulates the message length in bits (128 bits wide, enough for either SHA-2 length field).

## 5.4 Padding and length encoding

When the final chunk has been absorbed and fewer bytes than one block remain, the controller builds the padded block in place using constants that are shifted to

| Algorithm | First padding block | Extra block needed when | Final block |
|---|---|---|---|
| SHA-256 | 0x80 at byte fill, zeros, 64-bit length | fill >= 56 bytes | S_PAD, or S_PAD2 if no room |
| SHA-512 | 0x80 at byte fill, zeros, 128-bit length | fill >= 112 bytes | S_PAD, or S_PAD2 if no room |
| SHA3-256 | 0x06 at byte fill, 0x80 at byte 135 | never (OR merges to 0x86 at fill = 135) | S_PAD only |

*Table 5. Padding rules implemented by the controller*

For SHA3 the two padding bytes are merged with a simple OR. When the message leaves exactly one free byte (fill = 135) the markers collide in the same byte and combine to 0x86, which is the correct FIPS 202 behaviour. For SHA-2 the length field is placed at the end of the block by shifting `total` left by `len_sh`. If the 0x80 marker and the length do not both fit, the engine inserts a second block (S_PAD2) that contains only the length.

## 5.5 Controller state machine

The controller is a seven-state FSM in one clocked process with asynchronous reset. Figure 3 shows the transitions and Table 6 describes the states. The FSM is the sole source of core start pulses, so a core can never receive a start while it is busy.

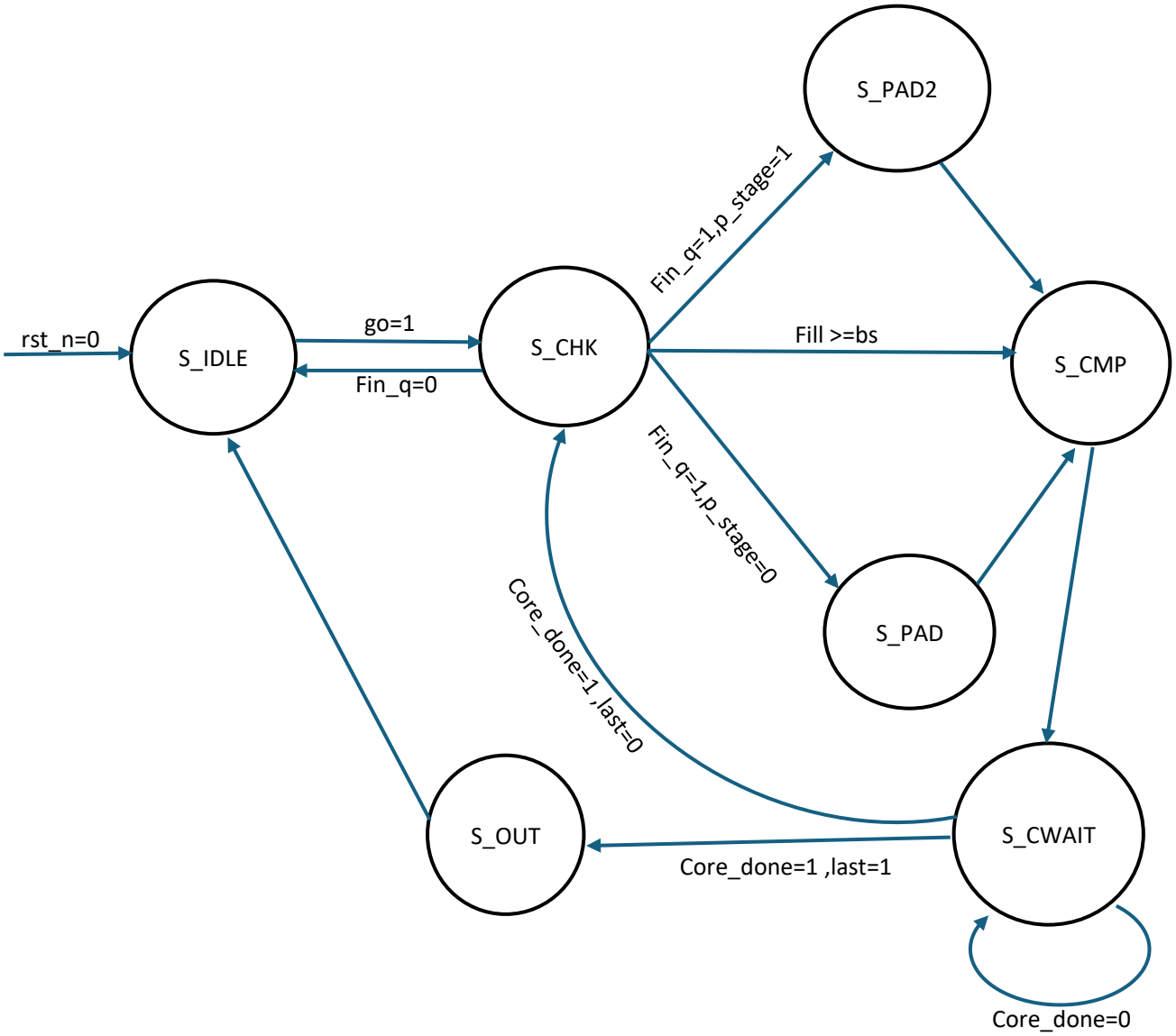



*Figure 3. State machine of hash_engine*

| State | Description | Next State & Transition Logic |
|---|---|---|
| S_IDLE | Waits for user input. Accepts init_cmd to configure alg and reset internal state, or go to absorb message data. | When go = 1 next state will be S_CHK. |
| S_CHK | Evaluates current byte occupancy (fill) to determine if a full block is available or if padding is required. | When fill become more or equal >= BS next state will be S_COMP.<br>if fin_q = 1 (Padding needed) and p_stage =0 (there is empty space) next state will be S_PAD.<br>if fin_q = 1 (Padding needed) and p_stage =1 (there is no empty space) next state will be S_PAD2.<br>Otherwise (Needs more data) next state is S_IDLE. |
| S_COMP | Initiates computation on the selected iterative compression core. | Always S_CWAIT. |
| S_CWAIT | Monitors compression core status until block processing completes. | when core_done = 1 AND last = 1 next state will be S_OUT.<br>when core_done = 1 AND last = 0 next state will be S_CHK.<br>when core_done = 0 next state will be S_CWAIT. |
| S_PAD | Constructs the first padded block (0x80 for SHA-2 or 0x06/0x80 for SHA-3) and inserts the length field if space allows. | Always next state is S_COMP. |
| S_PAD2 | Writes the SHA-2 length-only block. | Always next state is S_COMP. |
| S_OUT | Copies the formatted digest to the output register, clears inited, pulses done. | Always next state is S_IDLE. |

*Table 6. FSM states of hash_engine*

## 5.6 Chaining, digest formatting and error handling

The flag `first` is 1 until the first block of a message has been compressed. It drives the cores as `cont = !first` for SHA-256 and SHA-512 (so the first block starts from the standard initial value and later blocks start from the previous digest) and as `init = first` for SHA3-256 (so the first block starts from a zero state and later blocks are XORed into the retained state). A combinational multiplexer `dig_out` selects the digest of the active algorithm and left-aligns 256-bit results. The output register is loaded only in S_OUT, which makes the output stable and avoids exposing the SHA3 core's continuously changing state.

Errors are reported with a one-cycle done together with err (err then stays high until the next accepted command or the end of a message): an init_cmd with alg = 0, a chunk submitted without a valid session, or a chunk length above 64 bytes. Commands that arrive while the FSM is busy are ignored, and `busy` is the only indication to the host.

# 6. Compression Core Designs

## 6.1 SHA-256 core

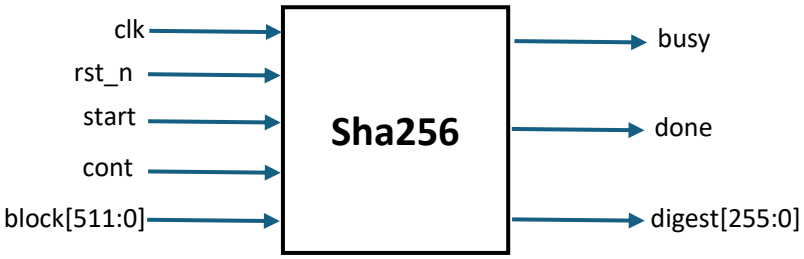



*Figure 4 Top-level I/O Block Diagram of the SHA-256 Core*

The core accepts one pre-padded 512-bit block per start request, executes one round per clock cycle for 64 cycles, and adds a finalisation cycle that folds the working variables into the chaining value. Its ports are start, cont, a 512-bit block, busy, done and a 256-bit digest. Input word 0 is block[511:480], and `cont` selects between the standard initial value H256 and the current digest.

**Datapath.** The core is an iterative design built around eight 32-bit working registers a to h, a sixteen-word window W[0:15] that serves as both input buffer and message schedule, a saved copy of the starting chaining value (`hprev`), and the digest register. The 64 round constants are a combinational case function indexed by t[5:0], so no ROM is required.

**Message schedule.** Instead of storing 64 words, the core uses the window as a shift register. During rounds 0 to 15 the current word is W[0]. From round 16 the new word is sigma1(W[14]) + W[9] + sigma0(W[1]) + W[0], the FIPS 180-4 recurrence. On every active cycle the window shifts down by one and the word `wt` enters at W[15].

**Round function.** Each cycle computes T1 = h + Sigma1(e) + Ch(e,f,g) + K[t] + W(t) and T2 = Sigma0(a) + Maj(a,b,c), then updates e = d + T1 and a = T1 + T2 while shifting the other variables down. Sigma1 rotates by 6, 11, 25 and Sigma0 by 2, 13, 22. Modulo-$2^{32}$ addition is the natural 32-bit truncation of Verilog wires.

**Control.** A 7-bit counter `t` and the flags busy, fin and done form three phases (Table 7). The counter is 7 bits wide so that it can reach 64 (0x40), the value seen in the waveform after the last round; the constant function uses only t[5:0].

| Phase | Condition | Action at the clock edge |
|---|---|---|
| Load | `start && !busy` | busy = 1, t = 0. a..h and hprev loaded from iv_sel (H256 or digest). W[0..15] loaded from block. |
| Rounds | `busy && !fin` | One round per cycle, W window shifts, t increments. fin is set when t == 63. |
| Finalise | `fin` | digest = hprev + {a..h} word by word. busy = 0, done = 1, fin = 0. |

*Table 7. Control phases shared by sha256_core and sha512_core*

### 6.1.1 Control FSM of sha256_core

The control of `sha256_core` is a three-state finite state machine (Figure 5). It has no separate state register: the state is encoded by the flags `busy` and `fin`, and the combination `busy`=0, `fin`=1 cannot occur. The asynchronous reset `rst_n` forces the machine into IDLE. The branches of the clocked process are evaluated in a fixed priority order (`start && !busy`, then `busy && !fin`, then `fin`), so a `start` pulse that arrives while the core is busy is ignored.

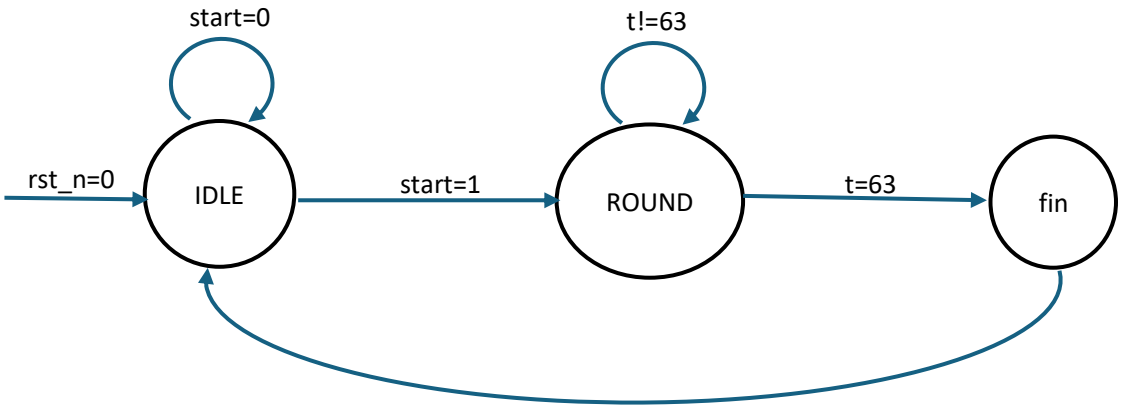



*Figure 5. Control FSM of sha256_core*

| State | Flags (busy, fin) | Actions at the clock edge | Next state |
|---|---|---|---|
| **IDLE** | 0, 0 | Waits for `start`. On `start`: `busy`=1, `fin`=0, `t`=0; `hprev` and `a..h` are loaded from `iv_sel` (H256 when `cont`=0, the previous `digest` when `cont`=1); `W[0..15]` are loaded from `block`. | When start=1 next state will be ROUND. |
| **ROUND** | 1, 0 | One round per clock: `h..b` shift down, `e`=`d`+T1, `a`=T1+T2, the `W` window shifts and `wt` enters `W[15]`, and `t` is incremented. `fin` is set by the edge that executes round 63. | when t =63 next state will be FIN. Otherwise stay in ROUND (64 cycles in total). |
| **FIN** | 1, 1 | `digest` = `hprev` + {`a..h`}, added word by word modulo 2^32. `fin`=0, `busy`=0 and `done`=1. | Next state always IDLE. |

*Table 8. States of the sha256_core control FSM*

Counted from the clock edge that samples `start`, edge 1 performs the load (IDLE to ROUND), edges 2 to 65 execute the 64 rounds (`t` = 0 to 63), and edge 66 performs the finalisation (FIN to IDLE). The block therefore takes 66 cycles. `done` is a registered one-cycle pulse: its default value is 0 on every edge and it is set only by the FIN to IDLE transition, so it is high during the first cycle back in IDLE, when `digest` is valid. `busy` stays high through FIN and drops on the same edge that raises `done`. The counter `t` ends at 64 (0x40) and keeps that value until the next load clears it, which is the value seen in the waveform after the last round.

## 6.2 SHA-512 core

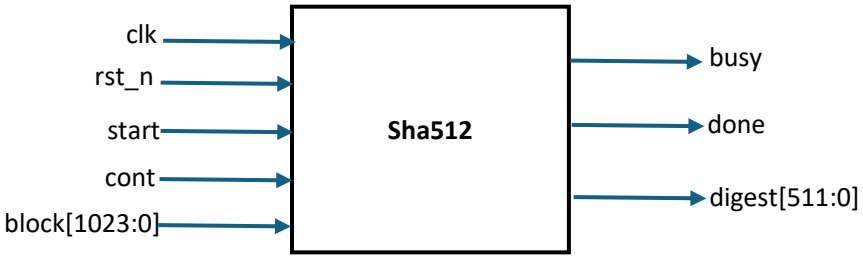



*Figure 6 Top-level I/O Block Diagram of the SHA-512 Core*

sha512_core applies the same architecture to 64-bit words: a 1024-bit block, 80 rounds, a 512-bit digest and an 80-entry constant table indexed by t[6:0].The counter is 8 bits wide and fin is set when t == 79, so the 80th round is the last. Table 12 summarises everything that differs from the SHA-256 core.

| Feature | sha256_core | sha512_core |
|---|---|---|
| Word width | 32 bits | 64 bits |
| Block input | 512 bits, word 0 = block[511:480] | 1024 bits, word 0 = block[1023:960] |
| Digest | 256 bits | 512 bits |
| Initial value | H256 (6a09e667 ... 5be0cd19) | IV512 (6a09e667f3bcc908 ... 5be0cd19137e2179) |
| Sigma0 / Sigma1 rotations | 2, 13, 22 / 6, 11, 25 | 28, 34, 39 / 14, 18, 41 |
| sigma0 / sigma1 | ROTR7, ROTR18, SHR3 / ROTR17, ROTR19, SHR10 | ROTR1, ROTR8, SHR7 / ROTR19, ROTR61, SHR6 |
| Rounds / counter | 64 / 7 bits, fin at t == 63 | 80 / 8 bits, fin at t == 79 |
| Constant table | K[0..63] on t[5:0] | K[0..79] on t[6:0] |
| Cycles per block | 1 + 64 + 1 = 66 | 1 + 80 + 1 = 82 |

*Table 9. Differences between the two SHA-2 cores*

### 6.2.1 Control FSM of sha512_core

The control of `sha512_core` is a three-state finite state machine (Figure 7). It has no separate state register: the state is encoded by the flags `busy` and `fin`, and the combination `busy`=0, `fin`=1 cannot occur. The asynchronous reset `rst_n` forces the machine into IDLE. The branches of the clocked process are evaluated in a fixed priority order (`start && !busy`, then `busy && !fin`, then `fin`), so a `start` pulse that arrives while the core is busy is ignored.

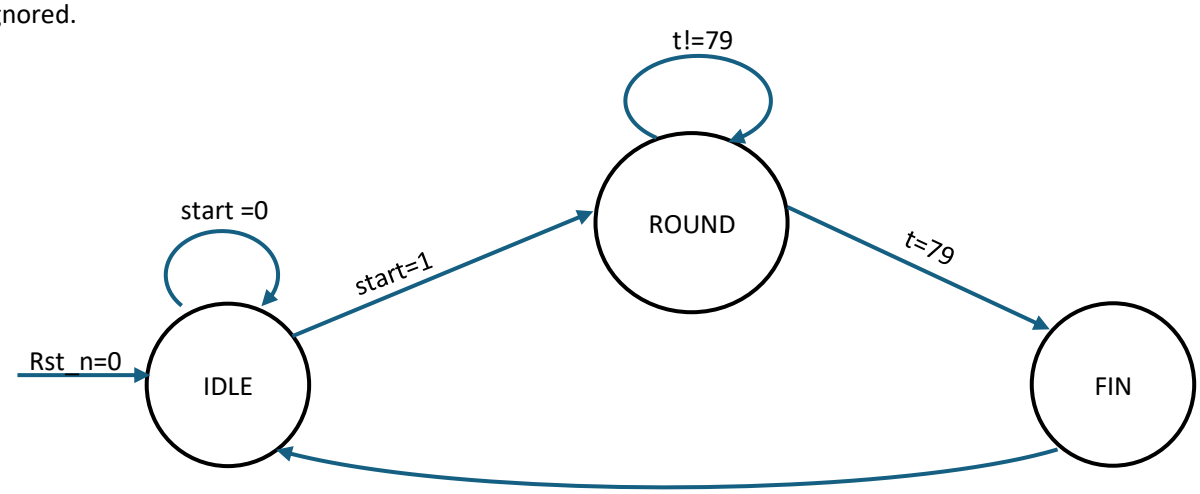



*Figure 7. Control FSM of sha512_core*

| State | Flags (busy, fin) | Actions at the clock edge | Next state |
|---|---|---|---|
| IDLE | 0, 0 | Waits for `start`. On `start`: `busy`=1, `fin`=0, `t`=0; `hprev` and `a..h` are loaded from `iv_sel` (IV512 when `cont`=0, the previous `digest` when `cont`=1); `W[0..15]` are loaded from `block`. | when start = 1 next state will be ROUND. Otherwise stay in IDLE. |
| ROUND | 1, 0 | One round per clock: `h..b` shift down, `e`=`d`+T1, `a`=T1+T2, the `W` window shifts and `wt` enters `W[15]`, and `t` is incremented. `fin` is set by the edge that executes round 79. | when t=79 next state will be FIN. Otherwise stay in ROUND (80 cycles in total). |
| FIN | 1, 1 | `digest` = `hprev` + {`a..h`}, added word by word modulo 2^64. `fin`=0, `busy`=0 and `done`=1. | Next state always IDLE. |

*Table 10. States of the sha512_core control FSM*

Counted from the clock edge that samples `start`, edge 1 performs the load (IDLE to ROUND), edges 2 to 81 execute the 80 rounds (`t` = 0 to 79), and edge 82 performs the finalisation (FIN to IDLE). The block therefore takes 82 cycles. `done` is a registered one-cycle pulse: its default value is 0 on every edge and it is set only by the FIN to IDLE transition, so it is high during the first cycle back in IDLE, when `digest` is valid. `busy` stays high through FIN and drops on the same edge that raises `done`. The counter `t` ends at 80 (0x50) and keeps that value until the next load clears it, which is the value seen in the waveform after the last round.

The FSM is identical to that of `sha256_core`; only the terminal count of the ROUND state (79 instead of 63), the width of `t` (8 instead of 7 bits) and the word width change.

## 6.3 SHA3-256 core

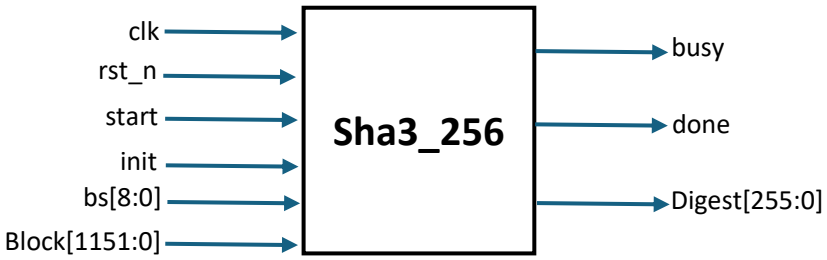



*Figure 8 Top-level I/O Block Diagram of the SHA3-256 Core*

The core implements Keccak-f[1600] as a sponge with a rate of 136 bytes and a capacity of 64 bytes, one permutation round per clock. A block is absorbed in one cycle and the 24 rounds follow in the next 24 cycles. The interface has a start/busy/done handshake, an `init` input (1 zeroes the state before absorbing, 0 continues from the current state), a 9-bit bs input (rate in bytes, 136 for SHA3-256), a 1152-bit block port of which only the first bs bytes are used, and a registered 256-bit digest.

**State and byte order.** The 1600-bit register `sq` holds 25 little-endian 64-bit lanes (lane n is sq[64n +: 64], byte i is sq[8i +: 8]), as defined in FIPS 202. The input block is big-endian, with byte 0 at block[1151:1144], so the absorb function XORs state byte i with block[1151-8i -: 8]. On the last round the digest register is loaded byte by byte with digest[255-8i -: 8] = final_state[8i +: 8], so the output reads as a conventional hexadecimal string. The register changes only on the last round of a permutation and is meaningful when done is high.

**Round function.** The function `round()` is fully combinational. *Theta* computes five column parities C[x] and D[x] = C[x-1] XOR ROL(C[x+1], 1). *Rho* and *pi* are merged into a table of 24 constant rotations written as bit-slice

concatenations, so they cost no gates. *Chi* uses 25 lane-wide AND-NOT-XOR expressions, and iota XORs a constant from the 24-entry function `rc()` into lane 0. The critical path is the theta XOR tree, the *chi* gate and the *Iota* XOR, which is short.

### 6.3.1 Control FSM of sha3_256_core

The control of `sha3_256_core` is a two-state machine (Figure 9). The state is the flag `busy` (IDLE when 0, PERM when 1) and the round counter `rnd` counts the 24 permutation rounds. There is no finalisation state, because the digest is taken from the combinational round output `final_state` on the same clock edge that executes the last round. The branch `start && !busy` has priority over `busy`, so a `start` pulse during a permutation is ignored.

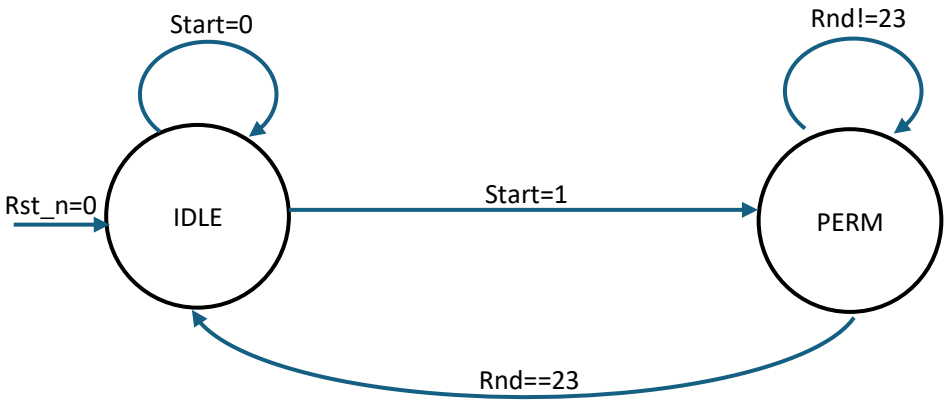



Figure 9. Control FSM of sha3_256_core

| State | Flags (busy) | Actions at the clock edge | Next state |
|---|---|---|---|
| IDLE | 0 | Waits for `start`. On `start`: `sq` = `absorb`(`init` ? 0 : `sq`, `block`, `bs`), which XORs the first `bs` bytes of the block into the state, `rnd`=0 and `busy`=1. With no `start` the state `sq` is retained, which is how multi-block messages are chained. | when start=1 next state will be PERM. Otherwise stay in IDLE. |
| PERM | 1 | `sq` = `round`(`sq`, `rnd`), one Keccak-f round per clock. While `rnd` != 23, `rnd` is incremented. On `rnd`==23 the new state is also copied into the digest register (digest[255-8i -: 8] = final_state[8i +: 8] for i = 0 to 31, the first 32 state bytes, most-significant byte first), `busy`=0 and `done`=1. | when rnd=23 next state will be IDLE. Otherwise stay in PERM (24 cycles in total). |

*Table 11. States of the sha3_256_core control FSM*

Edge 1, the one that samples `start`, absorbs the block (IDLE to PERM). Edges 2 to 25 apply rounds 0 to 23, and the last of them returns the machine to IDLE, so a block takes 25 cycles. `done` is a registered one-cycle pulse that is high in the first IDLE cycle, when `digest` is valid. Compared with the SHA-2 cores, this FSM has one state fewer and needs no feed-forward addition, which is why the SHA3 core finishes a block in 25 cycles instead of 66 or 82.

# 7. Verification and Simulation Results

## 7.1 Testbench and test vectors

tb_hash_engine instantiates hash_engine as the device under test, generates a 10 ns clock and applies a short asynchronous reset. For each algorithm it issues an init_cmd, waits for done, submits the message as a single final chunk, waits for done again, and compares the top bits of `digest` with a stored expected value. The message is

the seven-byte string "algeria" (0x616c6765726961), left-aligned by the testbench before submission. The test order is SHA-256, SHA-512, SHA3-256, and each wait has a 2000-cycle timeout. The VCD is written to tb_hash_engine.vcd, and 360 extra clocks hold the final values in the waveform.

| Algorithm | Expected digest for "algeria" (also seen in the waveform) |
|---|---|
| SHA-256 | `d854169a7fa8233ea9a1db6b5c249fa4919523e0adefb8cb72389d29db83b32a` |
| SHA-512 | `0c8cf817353e62e4e1aacf2b2f07b66901f0e43af65827cce4ba00e4edb8aab2`<br>`2c635a87c55bebe79c77290f813b5bac31790888d862ba6dadc966117a047e6a` |
| SHA3-256 | `f801565a9bf95564f1faee3c67fe9af5496d48d70f65b524294c7fe4f9d42b0c` |

*Table 17. Expected digests used by the testbench*

## 7.2 SHA-256 waveform

Figure 6 shows the signals of the instance u_s256: clk, start, busy, t, fin, a, e, hprev, wt, done and digest.

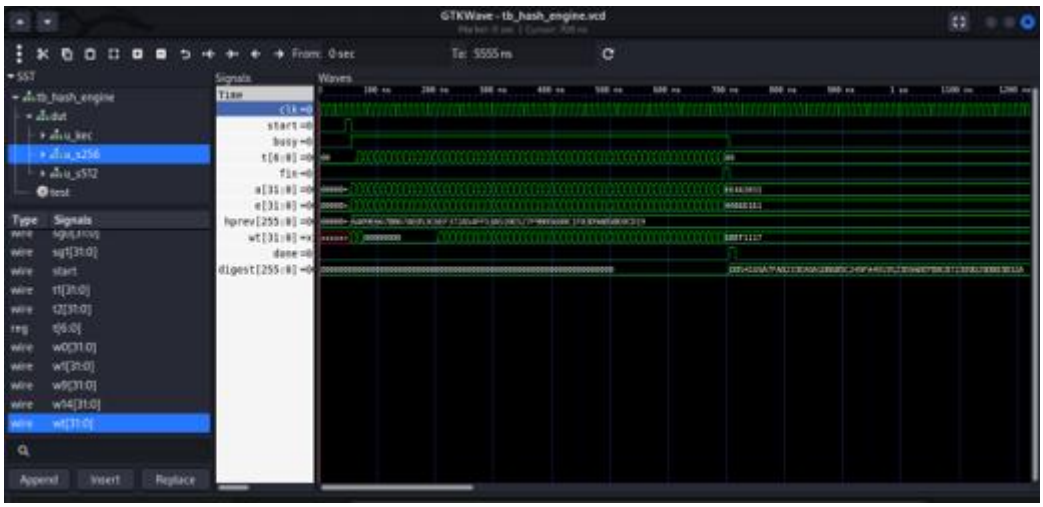



*Figure 10. GTKWave capture of u_s256 inside the hash_engine testbench*

- **Start and load.** A single start pulse occurs at roughly 40 ns and busy rises on the next clock edge, confirming that the load phase is registered.
- **Rounds.** The counter t advances once per clock from 00 to 40 (hex, i.e. 64) while a and e change every cycle, showing one round per clock.
- **Initial value.** hprev holds 6A09E667 BB67AE85 3C6EF372 A54FF53A 510E527F 9B05688C 1F83D9AB 5BE0CD19, exactly H256, so cont was low (first block).
- **Completion.** busy falls and done pulses at about 705 ns while the digest updates. The last cycle shows a = 6E4A3033, e = 4086D161 and wt = 6BEF1117 at t = 0x40.
- **Result.** The digest changes from zero to D854169A 7FA8233E A9A1DB6B 5C249FA4 919523E0 ADEFB8CB 72389D29 DB83B32A, which equals the expected SHA-256 value for "algeria".

## 7.3 SHA-512 waveform

Figure 7 shows u_s512 in the window 700 ns to 5555 ns, with clk, start, busy, t, fin, a, e, wt, hprev, done and digest.

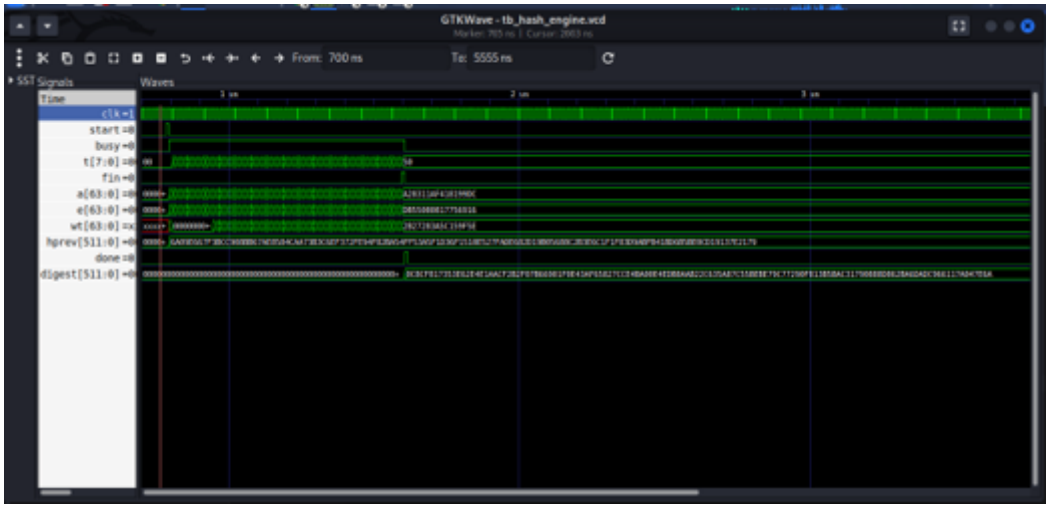



*Figure 11. GTKWave capture of u_s512 inside the hash_engine testbench*

- **Control flow.** A single-cycle start is accepted while idle and busy asserts on the next clock. The counter t runs from 00 to 50 hex (80 rounds) and keeps 0x50 afterwards, because t is cleared only on the next start.
- **fin and done.** fin is high for exactly one cycle, immediately before busy falls, matching the t == 79 condition. done pulses once together with the digest update.
- **Initial value.** hprev holds 6A09E667F3BCC908... throughout, so the block started with cont = 0.

**Digest.** It changes from 0 to 0c8cf817353e62e4e1aacf2b2f07b66901f0e43af65827cce4ba00e4edb8aab2

- 2c635a87c55bebe79c77290f813b5bac31790888d862ba6dadc966117a047e6a equal to the expected SHA-512 value.

## 7.4 SHA3-256 waveform

Figure 8 shows the window from about 1650 ns to 3300 ns with clk, init, start, busy, done, rnd[4:0], the state sq[1599:0] and digest[255:0].

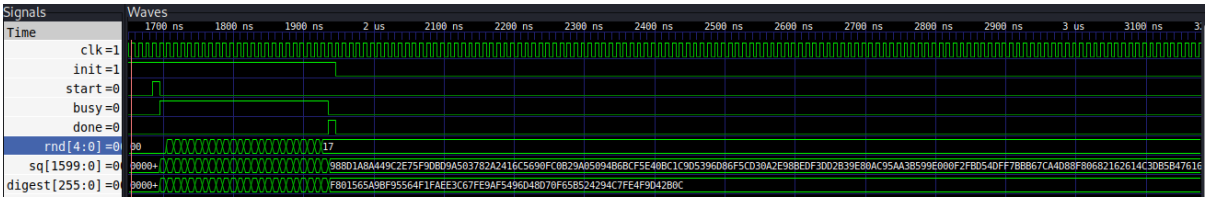



*Figure 12. GTKWave capture of the SHA3-256 core showing one complete block operation*

- **start and init.** A single short pulse near 1700 ns triggers the operation. With init high, the state is cleared and the block absorbed on that edge.
- **busy and rnd.** busy rises on the next edge and falls at roughly 1950 ns, a width of about 24 clock periods. rnd counts through the round indices and stops at hex 17 (decimal 23), which is the terminating condition in the RTL.
- **done.** A one-cycle pulse coincident with the falling edge of busy marks the instant the digest becomes valid.
- **sq and digest.** sq changes every cycle while busy is high and freezes once rnd reaches 23. The digest register is loaded once, on the last round (in this capture it is shown following sq, because the capture predates the registered digest). Both hold their value until the next start.
- **Result.** The final digest, F801565A9BF95564F1FAEE3C67FE9AF5496D48D70F65B524294C7FE4F9D42B0C, matches the testbench value for "algeria" and the hashlib result.

# 8. Conclusion

The design is a compact, standards-based hashing subsystem. hash_engine top control the three algorithms behind one streaming interface, with a clear seven-state controller, careful byte-level padding and a stable output register. The three cores are small iterative designs that need no memories: 66, 82 and 25 cycles per block.

The simulation evidence is consistent with the RTL in all three cases. Each waveform shows the expected load, round count, single-cycle done pulse and held digest, and the digests the independent computation for the message "algeria".

While functional correctness has been verified through RTL simulation, the immediate next phase of development focuses on target-specific logic synthesis, performance benchmarking, and physical resource analysis. Logic synthesis will map the Verilog implementations of hash_engine and its three underlying compression cores (sha256_core, sha512_core, and sha3_256_core) onto concrete ASIC standard-cell libraries or FPGA . This step is critical for obtaining precise hardware metrics, including total silicon area, flip-flop and LUT utilization, maximum achievable clock frequency ($f_{max}$), and total power dissipation. Furthermore, performance and resource analysis will allow for empirical evaluation of critical timing paths—such as the SHA-2 round addition trees enabling targeted architectural optimization before smart card silicon integration.